

ECE 398
Final Report

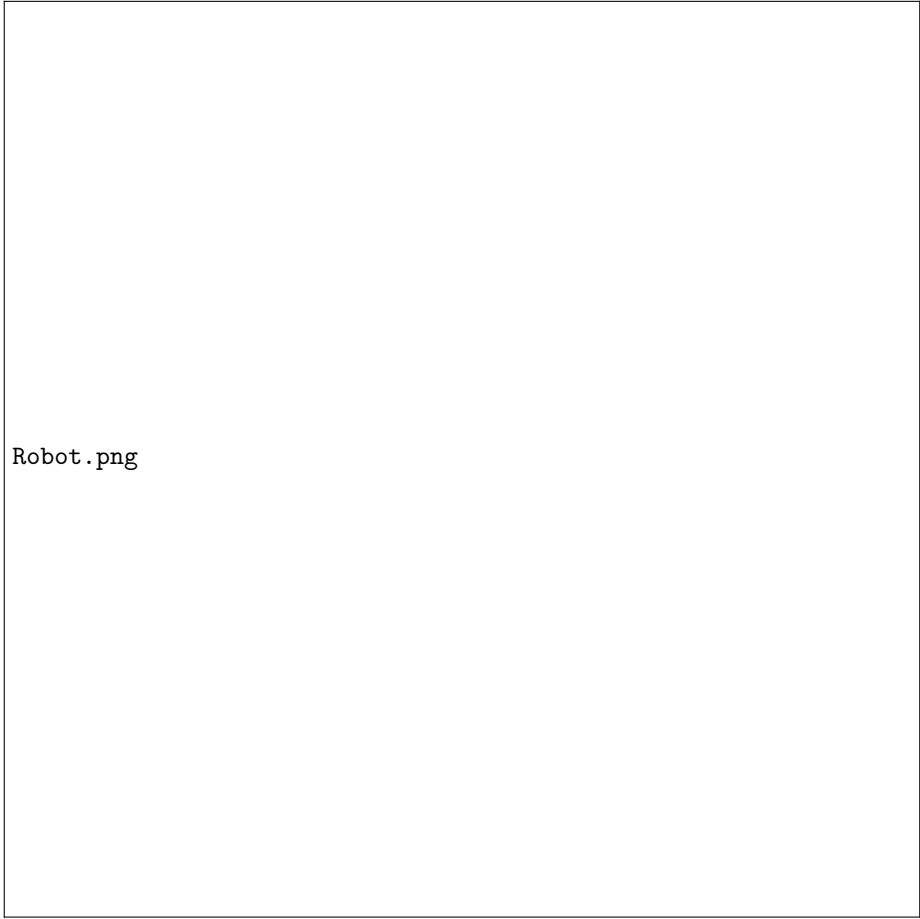
Development of a Quadrupedal Robot

Haruto Iguchi, Steve Xing

University of Illinois at Urbana-Champaign
December, 2024

Table of Contents

1. Introduction
2. Timeline
3. Roles
4. Methodology
5. Results
6. Discussions
7. Future Work
8. References



Robot.png

Introduction

We are developing a 12-legged quadrupedal robotic platform inspired by the Open Dynamic Robot Initiative, with the future goal of achieving dynamic walking capabilities suitable for real-world applications in agriculture, law enforcement, and search and rescue operations. Our hardware implementation includes six ODrive V3.6 motor controller boards and a dual CAN bus system to manage 12 BLDC motors efficiently across the robot's front and back actuators. This semester, our primary objectives have been to:

- Implement and refine a pre-programmed gait algorithm, enabling stable locomotion
- Testing the robot's performance limits to evaluate and tune to its full potential.

Project Timeline

January - August 2024

- Designed CAD models for a robot inspired by ODRI's SOLO robot [1].
- Fabricated components necessary for developing the robot, including bearings, belts, ODrives, CAN-buses, etc.

September 2024

- Assembled the robot, including wiring, crimping, and integrating the CAN bus, as well as ODrive controllers and Lithium Polymer batteries.
- Developed a CAN-web interface debugger to aid with calibrating the motors, e-stops, and tuning PID. (Figure 1)

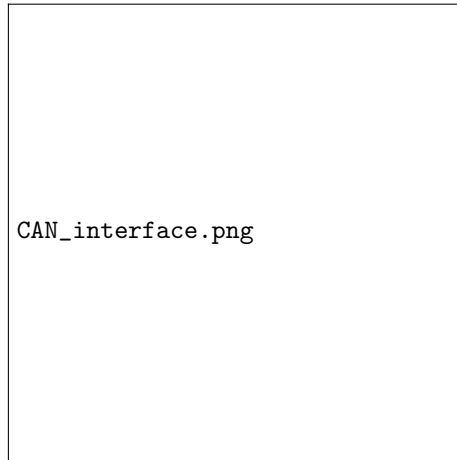


Figure 1: CAN-web interface.

October 2024

- Resolved hardware issues such as brittle motor wires, faulty CAN connections, and encoder wiring faults.
- Implemented PID tuning into the web interface, and used ODrive tuning guide as reference to aid with tuning. [2]
- Achieved optimal tuning for preliminary stability testing.
- Began testing basic movements such as standing and supporting its own weight, squatting motion where all four legs move up and down synchronously, as well as sequential leg lifting for balance tests.

November 2024

- Began researching potential gaits and locomotion algorithms such as Stanford Pupper. [3]
- Designed various gait based on experimental data. (Figure 2, Figure 3)

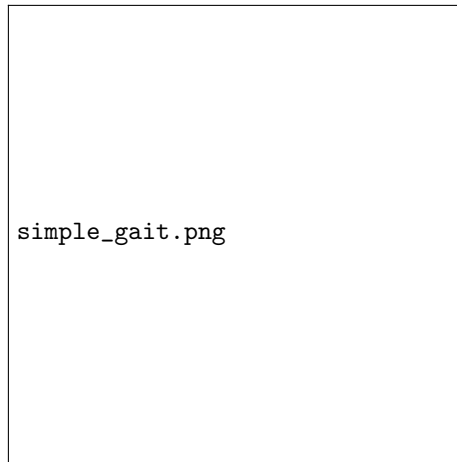


Figure 2: Simple sinusoidal gait

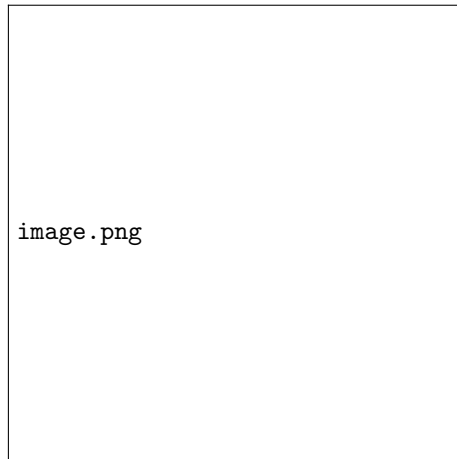


Figure 3: Parameterized gait

December 2024

- Performed tests to evaluate performance, which included actuator impulse and force response, PID tuning test to test tuning parameters under load, and maximum stall torque.

Roles

Steve

- Developed debugging tools for system diagnostics and calibration.
- Handled software implementation to translate mathematical gait models into executable Python commands.

Haruto

- Designed CAD models for the robot's structure and actuators.
- Debugged ODrive motor controllers and wiring issues.
- Developed mathematical models for gait optimization and performance evaluation.

Methodology

The development of our quadrupedal robot followed a systematic approach, where we design the hardware, develop software based on the hardware, perform tests to address challenges, and refine or troubleshoot until we reach a desired outcome.

Hardware Design

The hardware design primarily was based on the Open Dynamic Robot Initiative (ODRI), with a few minor tweaks. In contrast to the ODRI robot, our robot utilized six ODrive motor controller, each responsible controlling two motors. Two CAN buses act as the primary communication protocol between the computer and the ODrives, and 12 BLDC motors were used. Each motor drives a single joint allowing for a total of 12 DOF. The joints were driven by a timing system with a 9:1 gear reduction.

The main material used for the robot was PLA due to its high strength and light weight properties. For parts which encounter larger torque such as the timing pulleys (Figure 4) were outsourced to Chinese manufacturers to be printed out of Polycarbonate. Because the diameters of these pulleys are 9mm and 26mm, a material far stronger than PLA was required.

In addition to the materials, various size bearings were used to ensure that the robot stayed rigid.

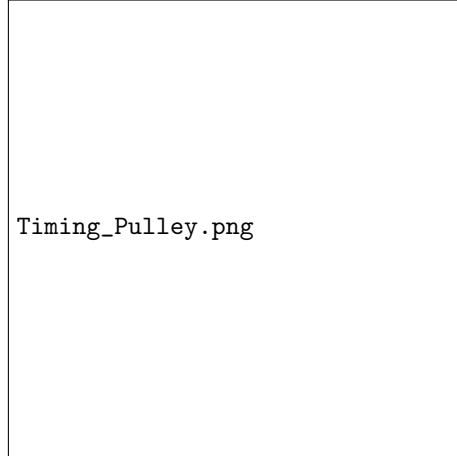


Figure 4: Timing Pulley

Software Tools

The development of software, which integrated the robot's movement as well as various debugging tools, were done in Python. The language was chosen primarily because of CAN libraries and ODrive libraries that exist in the ecosystem. A CAN-based web interface debugger was developed to help streamline the calibration sequence as well as the testing process. This tool allowed us to view

the position setpoint as well as the current position and the current flowing into the motors (Figure 5), which allowed us to better view the quality of the tuning parameters. The program also features a e-stop function which shuts off the motors.

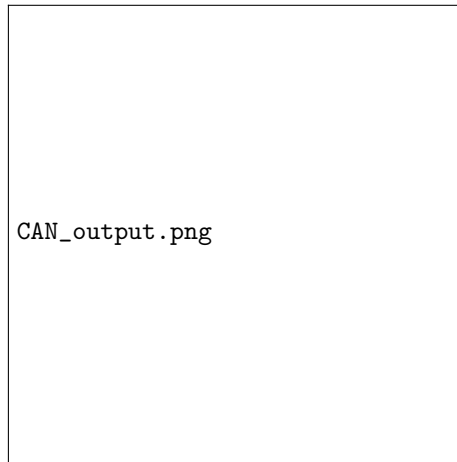


Figure 5: CAN-based web interface debugger output

Challenges

We encountered various issues with both the software and hardware development of the robot. Since the majority of the code was recycled from previous versions of the robot, we initially believed that initializing the CAN protocol and other software was relatively trivial. However, we encountered compatibility issues stemming from using different machines, different motor-related parameters.

An example of an ongoing challenge is that of stability. We are still unsure as to why the robot experiences difficulty when attempting to walk, however, we believe that this is due to the compliance of the robot. As there are some flexibility to the body of the robot.

Another challenge is the Polycarbonate pulleys (Figure 6). Although the pulley stay intact the majority of the time, there have been a few instances where the teeth of the pulleys wears off as a result of stress. As such, we are hoping to switch to CNC machined pulleys in the future.

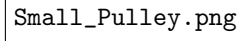
The image is a placeholder for a diagram of a pulley system. The text "Small_Pulley.png" is centered within a rectangular frame, indicating the location of the image file.

Figure 6: Pulley

Results

Three different tests were conducted to measure the performance of the actuator.

Test 1: PID Control Response

This test aimed to check the actuator's ability to track position setpoints under no external load, and how effectively it can return to the setpoint after an impulse or force. The upper graph (Figure 7) shows the setpoint (orange line) and the actual measured position (blue line). We see that after a force is applied to the actuator, the measured position shows spikes. We see similar spikes in the current graph as well. We also see that there are rectangular shapes in the graph where the actuator is forced to hold a position.

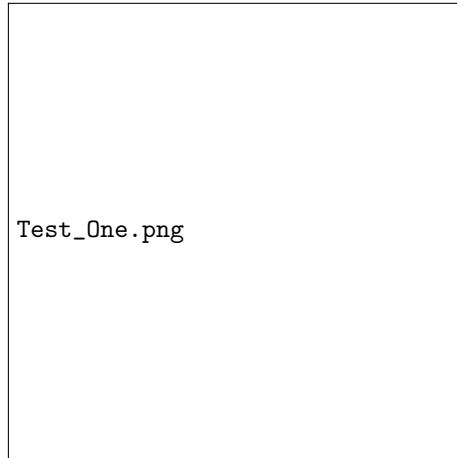


Figure 7: PID with impulse

Test 2: PID Control Response Under Load

In this test, we aimed to evaluate the PID parameters under a load. We conducted two tests: 0.6kg and 1.0kg (Figure 8). After each interval, we see a spike in the graph of the current as well as the position. As the actuator approaches an angle parallel to the ground, we see that it struggles to lift the weight. We set a current limit of 7.5 Amps on the actuator which is reflected in the bottom graph (Figure 8). As the current reaches the limit, the actuator stalls.

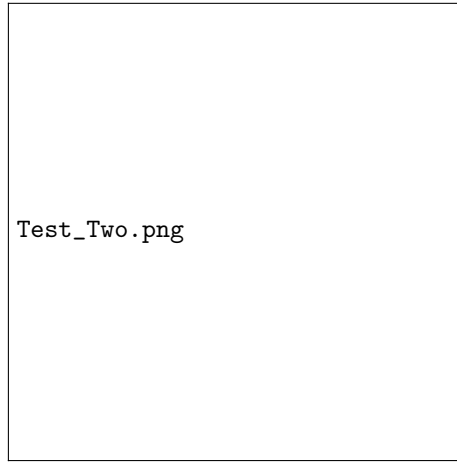


Figure 8: PID under load

Test 3: Torque Output Measurement

In the third test, the actuator was pressed against a scale at varying depths to attempt to measure the torque output (Figure 9). The upper graph shows the four pressing motions seen by the valleys between the peaks. As the actuator presses deeper into the scale, we also see the absolute current increases as well.

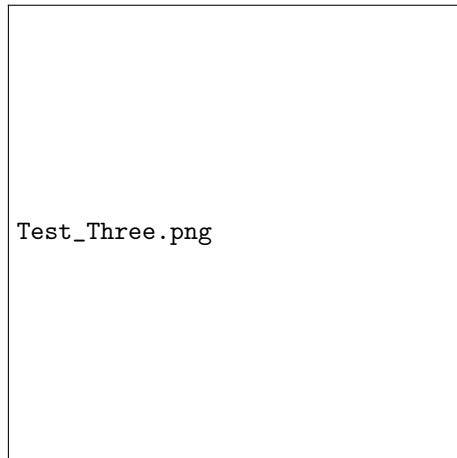


Figure 9: Torque Output Graph

Discussion

Test 1: PID Control Response

Referencing the position graph (Figure 7), the zero-position aligns with ~ 2.65 . Moving the end-effector up/down at varying speeds reveals two types of sudden changes:

- Spikes: Seen at 120-200 seconds, spikes indicate fast counter-clockwise changes, such as sudden elevation shifts causing actuator bending. Corresponding current spikes (8A) align with motor controller limits.
- Rectangles: At 220-240 seconds, flat current ($-7.5A$) represents forcing a position upon the motor. We see that as time progresses, the current also increases indicating that more force is applied to return to the setpoint.

Our future tests aim to correlate measured data with real-world conditions, particularly for walking scenarios.

Test 2: PID Control Response Under Load

In the graph (Figure 8), at 250-270 seconds, the system overshoots the setpoint significantly. Beyond 1.8 , the motor stalls due to reaching the current limit ($7.5A$). Observations suggest a weight margin of 0.8 kg (8 actuators $\times 0.6$ kg - 4 kg robot weight). Loads exceeding 1 kg risk overheating or system failure. Future work includes fine-tuning loads between 0.6 - 1.0 kg and exploring temporary overrides of current limits for short bursts.

Test 3: Torque Output Measurement

Five pressing motions highlight actuator response under varying depths. The early spikes (spike 1 at 150 seconds)(Figure 9) show consistent current draw and weight measurements (0.6 kg). Later spikes (spikes 3-5) max out at 1.0 kg, constrained by current limits ($-7.7A$). The system struggles to exceed certain depths due to physical constraints. The above findings confirm the impact of current limits on actuator performance and torque output.

Test 1-3 Takeaways

Current limits significantly influence the robot capabilities, affecting torque and stall conditions for the motors specifically. Our future tests should explore increasing current limits to assess the capabilities of the motor or see if our configuration results in restrictions. These results will help identify ideal gait speed and potential jumping capabilities.

Analysis of Walking

In the walking test video, the legs move vertically rather than following sinusoidal curves, differing vastly from standard "walking" algorithms featured in robots such as Stanford Doggo. Some unexpected behaviors include:

- Forward-shifting of the Center of Mass: Both left-back legs move simultaneously due to PID over-compliance, causing a "falling forward" effect.

- Uneven leg motion: Delayed leg movements cause exaggerated leaps.
- Hip bending: The right hip leans inward, destabilizing the robot. Throughout the video, legs lag behind hip pivots, further highlighting over-compliance.

Our future adjustments include locking hips into a rigid position to remove any errors resulting from their overcompliance, and refining PID tuning (e.g., reducing proportional and eliminating integral terms) to enhance stability and reduce compliance.

Future Work

The results gathered from this project provide a foundation to further advance the quadrupedal robot. Several improvements and directions have been identified to address the current limitations and enhance the robot's performance for potential real-world applications.

1. **Optimization of PID Control and Implementation fo Feedforward System**

The current PID controller demonstrates compliance, which may be beneficial for certain applications, however, when the robot is walking, it requires the system to be rigid so as to not sink during the gait. The ODrive allows for a simple yet effective solution of changing the torque constant. We also believe that implementing a feedforward system based on the difference between the setpoint and actual position may be beneficial.

2. **Increasing Actuator Torque and Current Limits**

Although the rated current for the motor is 7.5A, we are interested in potentially increasing the current limit past 7.5A in short bursts when high torque is necessary, such as certain portions of the gait. We believe that this may mitigate overheating issues as well as problems rooting from insufficient torque.

3. **Integration of Sensors for Environmental Feedback and Advanced Gait Algorithms**

In the future, we hope to integrate various sensors such as IMUs and RGB-D cameras to develop more advanced gaits. For instance, altering the length or height of the total gait to counter obstacles. We believe we can start by developing a fully parameterized gait.

Citation:

- [1] Open Dynamic Robot Initiative, “Open Dynamic Robot Initiative,” open-dynamic-robot-initiative.github.io. [Online]. Available: <https://open-dynamic-robot-initiative.github.io/>. [Accessed: Oct. 24, 2024].
- [2] ODrive Robotics, “ODrive Tuning Guide,” ODrive Documentation, Version 0.5.6. [Online]. Available: <https://docs.odriverobotics.com/v/0.5.6/control.html>. [Accessed: Oct. 24, 2024].
- [3] J. Smith, M. Seok, and H. Kim, “Online gait synthesis for quadrupedal robots,” arXiv preprint arXiv:1905.04254, May 2019. [Online]. Available: <https://arxiv.org/pdf/1905.04254>. [Accessed: Oct. 24, 2024].
- [4] M. A. Şen, V. Bakırcıoğlu, and M. Kalyoncu, “Inverse Kinematic Analysis of a Quadruped Robot,” International Journal of Scientific and Technology Research, vol. 6, no. 9, Oct. 2017. [Accessed: Oct. 24, 2024].
- [5] X. Zeng, S. Zhang, H. Zhang, X. Li, H. Zhou, and Y. Fu, “Leg Trajectory Planning for Quadruped Robots with High-Speed Trot Gait,” Applied Sciences, vol. 9, no. 7, pp. 1508–1508, Apr. 2019, doi: 10.3390/app9071508.